

Qudit LDPC distance finding and
bounding algorithms

Audit LDPC distance finding as an integer linear programming (ILP)

problem

An explanation of the ILP code in qudit LDPC.

The distance of a QEC code is the minimum weight of a logical operator (which is an undetectable error that exists outside of the space of stabilisers).

For qudits of prime dimension, an arbitrary Pauli operator can be represented by a vector:

$$x = (a | b) \in \mathbb{F}_q^{2n} \Rightarrow X^{a_1} Z^{b_1} \otimes \dots \otimes X^{a_n} Z^{b_n}$$

For two Pauli vectors $x = (a | b)$, $s = (c | d)$

their commutation relation is determined by the symplectic inner product

$$\langle s, x \rangle_{sp} = c \cdot b - d \cdot a \pmod q$$

They commute iff $\langle s, x \rangle_{sp} = 0 \pmod q$ * c.f. Symplectic product

For a stabiliser code with stabiliser generators

$$S_1, \dots, S_m$$

an error x is undetectable precisely when it commutes with every stabiliser $\langle S_j, x \rangle_{sp} = 0 \pmod q \quad \forall j$

Define the normaliser:

$$N(S) = \{x : \langle S_j, x \rangle_{sp} = 0 \quad \forall j\}$$
$$S = \{s : s|\psi\rangle = |\psi\rangle \quad \forall |\psi\rangle \in C\}$$

The distance of a stabiliser code is:

$$d = \min_{x \in N(S) \setminus S} wt(x)$$

where $wt(x) = |\{ \text{quits on which } x \text{ acts nontrivially} \}|$

$$\text{i.e.} \quad \sum_i 1[x_i \neq 0] = wt(x)$$

Now, consider that we have a logical operator l , which is NOT min weight.

If $\langle l, x \rangle_{sp} \neq 0$ but $\langle S_j, x \rangle_{sp} = 0 \quad \forall j$, then x must be a nontrivial logical operator. It is obviously not in the same equivalence class, it is the conjugate.

So for a fixed logical operator l , we may calculate a corresponding minimum weight conjugate logical operator by solving

$$\min_x wt(x) \quad \text{subject to} \quad \langle S_j, x \rangle_{sp} = 0 \quad \forall j$$
$$\langle l, x \rangle_{sp} \neq 0$$

So in the simplest of terms the ILP is:

$$\min_x \sum_{i=1}^n 1(x_i \neq 0)$$

$$\text{s.t. } Sx = 0 \pmod{q}$$

$$1x \neq 0 \pmod{q}$$

$$x_i \in \{0, \dots, q-1\}$$

Now translating this to a more tenable form:

$S \in \mathbb{Z}_q^{m \times n}$ which is the stabiliser matrix.

There are m stabilisers and n qubits.

We have $x \in \mathbb{Z}_q^n$ which gives the error on each of the n qubits and $x_i \in \{0, \dots, q-1\}$ s.t. $x_i \in \mathbb{Z}_q$

How to handle modular equations for ILP?

$$\text{We want } S_i x = 0 \pmod{q}$$

$$S_i x = 0 \pmod{q} \text{ iff } \exists k_j \text{ s.t. } S_i x = q k_j$$

$$\text{Therefore } S_i x - q k_j = 0$$

Now k_j must be represented using bounded integer variables

$$k_j = z_{j,1} + q z_{j,2} + q^2 z_{j,3} + \dots \text{ where } z_{j,n} \in \{0, \dots, q-1\}$$

This is just a base- q representation. * - why do it this way?

So the constraint is:

$$\sum_i S_{ji} x_i - q z_{j,1} - q^2 z_{j,2} - q^3 z_{j,3} - \dots = 0$$

$$\text{so } S_j x = 0 \pmod{p}$$

For q -LDPC codes, we have $\exists w$ s.t. $\text{wt}(S_j) \leq w \quad \forall j$

$$0 \leq S_{ji} \leq q-1, \quad 0 \leq x_i \leq q-1$$

$$\text{Hence: } 0 \leq S_{ji} x_i \leq (q-1)^2$$

If there are at most w nonzero terms in S_j

$$0 \leq S_j x \leq (q-1)^2 w$$

and we had $q k_j = S_j x$ so:

$$k_j = \frac{S_j x}{q} \Rightarrow 0 \leq k_j \leq \frac{(q-1)^2 w}{q}$$

So we choose

$$\left\lceil \frac{\log((q-1)^3 w)}{\log(q)} \right\rceil \quad * - \text{according to the code, I would think this should actually be}$$
$$\left\lceil \log((q-1)^3 w) - \log(q) \right\rceil$$

This is likely a mistake in the original code.

Logical constraint

$$lx \neq 0 \pmod{q} \quad (\text{this represents } \langle l, x \rangle_q \neq 0 \pmod{q})$$

Since x, l have integer representations in $0, \dots, q-1$

$$0 \leq lx \leq (q-1)^3 w \log$$

$$\text{So we have } lx - qk_\ell = r.$$

Instead of allowing

$$r \in \mathbb{Z}_0, \dots, q-1^3 \quad \text{we force } 1 \leq r \leq q-1$$

$$\text{so } 1 \leq lx - qk_\ell \leq q-1 \Rightarrow 1 \leq \sum_i l_i x_i - \sum_r q^r k_r \leq q-1$$

where we once again represent in base q

with $k_r \in \mathbb{Z}_0, \dots, q-1^3$. * - again why do it this way?

ILP objective to minimise / Hamming weight

Qudits vs qubits mean a distinction here.

For qubits,

$$x_i \in \{0, 1\}$$

$$\text{so } \text{wt}(x) = \sum_i x_i$$

But for qudits $x_i \in \{0, 1, \dots, q-1\}$ with $q > 2$

We are no longer minimising $\sum_i x_i$

because we want the smallest number of errors (of any $0 \dots q$) that leads to a logical operator

i.e. $\min \sum_i 1[x_i \neq 0]$ (add 1 to the weight for each non-zero $x_i \in \{1, 2, \dots, q-1\}$)

We do this using indicator variables like z

Introduce $y_{i,a}, \dots, y_{i,q-1} \in \{0, 1\}$

$$x_i = 1y_{i,1} + 2y_{i,2} + \dots + (q-1)y_{i,q-1}$$

and:

$$\sum_{a=1}^{q-1} y_{i,a} \leq 1$$

This means the allowed possibilities are:

$x_i = 0$ with all $y_s = 0$

$x_i \geq 1$ if $y_{i,1} = 1$

$x_i \in \{0, \dots, q-1\}$ with $y_{i,i} = 1$ (and rest 0).

$$\text{so } \sum_a y_{i,a} = \begin{cases} 0 & x_i = 0 \\ 1 & x_i \neq 0 \end{cases}$$

so we minimise:

$$\min \sum_{i=1}^n \sum_{a=1}^{q-1} y_{i,a} \quad \text{which represents the Hamming weight}$$

and we may recover x_i from this.

Thus we have overall:

$$\min_{x, y, z, k} \sum_{i=1}^n \sum_{a=1}^{q-1} y_{i,a}$$

$$\text{s.t. } x_i - \sum_{a=1}^{q-1} a y_{i,a} = 0 \quad \forall i, \quad (\text{this imposes } x_i = 1y_{i,1} + \dots + (q-1)y_{i,q-1})$$

$$\sum_a y_{i,a} \leq 1 \quad \forall i, \quad (\text{so that only one } y_{i,a} \text{ is at most 1})$$

$$\sum_i S_{i,0} x_i - \sum_r q^r z_r = 0 \quad \forall i \quad (\text{Impose } S_0 x = 0 \pmod{q})$$

$$1 \leq \sum_i l_i x_i - \sum_r q^r k_r \leq q-1 \quad (\text{Impose } lx \neq 0 \pmod{q})$$

$$x_i \in \{0, \dots, q^{-1}\},$$

$$y_{ia} \in \{0, 1\},$$

$$z_{jr}, k_r \in \{0, \dots, q^{-1}\}$$

$$R_s = \lceil \log_2 ((q-1)^3 w_{stab}) \rceil$$

$$R_e = \lceil \log_2 ((q-1)^3 w_{eq}) \rceil$$

Note on random stabilisers:

If we have a stabiliser g and logical l

$l' = l + g$ is the same logical as l .

So we can replace $l' \rightarrow l$ which are equivalent problems in QEC land but not for the TLP solver.

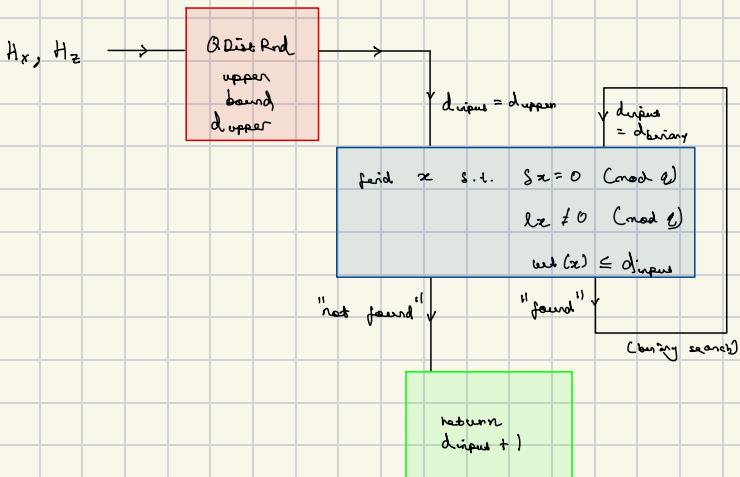
Alternative formulations

Feasibility MILP:

$$\begin{aligned} \text{find } x \quad \text{s.t.} \quad & Sx = 0 \pmod{q} \\ & Lx \neq 0 \pmod{q} \\ & \text{wt}(x) \leq d \end{aligned}$$

Given a heuristic d ,

So a potential pipeline could be:



I.e. count down from the heuristic distance upper bound until ILP can no longer find a solution, proving optimality.

BP-OSD distance upper bound and heuristic

We can use non-binary BP-OSD over $GF(q)$ to heuristically search for low-weight logical operators.

Suppose we have stabiliser matrix H and logical operator l .

We want a min-weight operator e satisfying:

$$He = 0 \pmod{q}$$

$$\text{and } \langle l, e \rangle \neq 0.$$

So our optimisation problem is $\min_e w(e)$ subject to $He = 0$ and $le = 1$.

Why is $le = 1$?

It doesn't need to be

$le = a$, $a \in \mathbb{F}_q$ would identify a valid conjugate logical

The code chooses 1 because for all prime q , nonzero values are equivalent upto multiplying e by a nonzero scalar.

This means we do:

$$H_{aug} = \begin{pmatrix} H \\ l' \end{pmatrix} \text{ where } l' = l + s \quad (\text{random stabiliser combo})$$

and $S_{\text{aug}} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}$

so $H_{\text{aug}} e = S_{\text{aug}}$ is what we force BP to decode

e gives us the heuristic min-weight logical.

This can be used to warm start the branch and bound ILP solver.

This is faster.